

Enhancing Social Media Evidence Collection and Reporting — FeedLens

Constraints: FeedLens is an automated forensic tool that streamlines the process of collecting and analyzing social media evidence, focusing primarily on Instagram. By leveraging automation, we aim to enhance the efficiency of evidence collection, reduce the potential for human error, and improve the overall quality of findings.

Project Report – CAP 797**Submitted To****Lovely Professional University**

**In partial fulfilment of the requirements for the award of degree
of Master of Computer Application**

**Submitted By,
Samrat Mandal
12319514**

**Supervised By,
Gaganpreet Kaur**



LOVELY
PROFESSIONAL
UNIVERSITY

**Lovely Professional University Phagwara,
Punjab 2025**

**Prepared For CAP797: Cyber Forensic Lab
Work Instructor: Gaganpreet Kaur**

To whom so ever it may concern

I, **SAMRAT MANDAL** (UID:12319514) hereby declare that the work done by me on " **Enhancing Social Media Evidence Collection and Reporting - FeedLens** " under "CAP797 coursework" from January 2025 to May 2025 at Lovely professional University, Phagwara, Punjab, is a record of original work for the partial fulfillment of the requirements for the of the degree Master of Computer Applications. The results embodied in this report have not been submitted to any other subject or university.

Submitted By

SAMRAT MANDAL

12319514

Prepared For CAP797: Cyber Forensics Lab Work

Instructor: **Gaganpreet Kaur**

ACKNOWLEDGEMENT

For all the efforts behind the paperwork & project, I first & for most would like to express my sincere appreciation to **Gaganpreet Kaur** (UID:28460), School of Computer Applications, for their extended help & suggestions at every stage of this project work. It is with a great sense of gratitude that I acknowledge the support, time to time suggestions and highly indebted to my guide. Finally, I pay sincere thanks to all those who indirectly and directly helped me towards the successful completion of the real time project "**Enhancing Social Media Evidence Collection and Reporting - FeedLens**" for Class Assessment work 4 and End Term Practical.

With due regards

SAMRAT MANDAL

12319514

CERTIFICATE

This is to certify that the project report titled "Enhancing Social Media Evidence Collection and Reporting – **FeedLens** " is being submitted by SAMRAT MANDAL (UID: 12319514), in MCA 4*semester is a record Bonafide work carried out by him. The results embodied in this report have not been submitted to any other University for the award of any degree.



Signature and Name of the Supervisor: Gaganpreet Kaur (UID: 28460)

Designation: Assistant Professor SCA

School of Computer Applications,

Lovely Professional University,

Phagwara, Punjab.

Remarks (if any):

Date:

Signature of Head of Department

School of Computer Applications,

Lovely Professional University,

Phagwara, Punjab.

Remarks (if any):

Date:

Document Approval

Signature	Approval Authority Name	Title	Date	Comments
	Gaganpreet Kaur	CA1	25/02/2025	
	Gaganpreet Kaur	CA2	8/04/2025	
	Gaganpreet Kaur	CA3	25/04/2025	
		ETP		

Table of Contents

Declaration	2
Acknowledgement	3
Certificate	4
Document Approval	5
Table of Contents	6
Abstract of Project	9
1. Introduction	10
1.1 Purpose	10
1.2 Objective	10
1.3 Document Conventions	11
1.4 Definitions, Acronyms, Abbreviations	11
2. System Analysis	11
2.1 Introduction	11
2.2 Existing System	11
2.3 Proposed System	12
2.4 Advantages of Proposed System	12
2.5 Operating Environment	13
2.6 Design & Implementation Constraints	13
3. Feasibility Study	14
3.1 Economic Feasibility	14
3.2 Technical Feasibility	15
3.3 Operational Feasibility	15

Enhancing Social Media Evidence Collection and Reporting – FeedLens

4. State of Art / Literature Review	16
4.1 Literature Review	16
4.2 Research Gap	17
5. System Specifications	17
5.1 Hardware Requirements	17
5.2 Software Requirements	18
6. Specific & Functional Interfaces	18
6.1 User Interfaces	18
6.2 Hardware Interfaces	19
6.3 Software Interfaces	19
6.4 Communication Interfaces	19
7. Project Description	19
7.1 Problem Definition	19
7.2 Project System Overview	20
7.3 Modules Overview & UML Analysis Models	20
8. System Design Diagrams	24
8.1 System Flow Diagrams	24
8.2 DFD Level 0	25
8.3 DFD Level 1	25
8.4 Entity Relationship Diagrams (ERDs)	26
8.5 Use Case Diagram	26
8.6 Activity Diagrams	27
9. FeedLens - Potential Future Enhancements	28
9.1 Android Version Development	28
9.2 Enhanced Platform-Specific Handling	28
9.3 Advanced OCR & Text Processing	29
9.4 AI-Powered Data Summarization & Analysis	30

Enhancing Social Media Evidence Collection and Reporting – FeedLens

9.5 Reporting & Export Enhancements	31
10. System Design Screen & Outputs	32
10.1 System UI	32
10.2 Output	33
11. Conclusion	35

Table of Figures

8. System Design Diagrams	24
8.1 System Flow Diagrams	24
8.2 DFD Level 0	25
8.3 DFD Level 1	25
8.4 Entity Relationship Diagrams (ERDs)	26
8.5 Use Case Diagram	26
8.6 Activity Diagrams	
10. System Design Screen & Outputs	32
10.1 System UI	32
10.2 Output	33

Abstract of Project

This report details the analysis of "FeedLens," a proposed software tool designed to address challenges faced during digital investigations involving social media accounts. The primary problem is the manual, time-consuming, and error-prone process of documenting evidence such as posts, messages, timelines, and account information from platforms like Facebook, Instagram, X (Twitter), etc. FeedLens aims to automate this process by utilizing browser automation (Selenium) to capture sequential screenshots during scrolling, optionally applying timestamp overlays, performing Optical Character Recognition (OCR) for text extraction, calculating file hashes (SHA-256) for integrity, and compiling all collected data into a comprehensive PDF report. The tool features a Graphical User Interface (GUI) built with Tkinter for ease of use by investigators. This analysis covers the system's objectives, feasibility, design, functionality, requirements, and identifies gaps between the initial problem statement and the current implementation.

1. Introduction

1.1 Purpose The purpose of the FeedLens project is to provide law enforcement and digital investigators with a standardized, efficient, and reliable method for capturing and documenting potentially relevant information from the web interfaces of social media accounts and other web pages belonging to suspects or individuals under investigation. It aims to streamline the creation of digital evidence records (akin to digital Panchama's), reduce the possibility of human error during manual capture, ensure thoroughness by automating scrolling, and provide verifiable data through hashing and timestamping.

1.2 Objective

- A. Develop a desktop application (initially for Windows/Desktop environments) with an intuitive GUI.
- B. Automate the process of navigating to specified web pages (including social media platforms).
- C. Implement automated scrolling (up and down) and sequential screenshot capture of web page content.
- D. Allow for single, on-demand screenshot captures.
- E. Provide options to configure capture settings (e.g., scroll delay, browser choice, image format).
- F. Integrate optional OCR capabilities to extract text from captured screenshots.
- G. Calculate and record SHA-256 hashes for each captured screenshot to aid integrity verification.
- H. Optionally overlay timestamps onto screenshots at the time of capture.
- I. Organize captured data (screenshots, extracted text, hashes) into session-specific folders.
- J. Generate comprehensive PDF reports summarizing the captured session, including case details, settings used, screenshots, extracted text, hashes, and an activity log.
- K. *Address (as per problem statement, though not fully implemented) the need for handling specific platforms like Facebook, Twitter, Instagram, Telegram, WhatsApp, Google accounts.*

L. Address (as per problem statement, though not implemented) the need for separate Windows and Android versions.

1.3 Document Conventions This report follows standard software documentation conventions. Sections are numbered hierarchically. Key terms and acronyms are defined. Code elements are referenced where applicable. UML (Unified Modeling Language) concepts are used conceptually in the relevant section.

1.4 Definitions, Acronyms, Abbreviations

- **FeedLens:** The name of the proposed software tool.
- **GUI:** Graphical User Interface.
- **OCR:** Optical Character Recognition.
- **SHA-256:** Secure Hash Algorithm 256-bit (a cryptographic hash function).
- **Selenium:** A suite of tools for automating web browsers.
- **WebDriver:** An API and protocol that allows programs to control a web browser.
- **Panchnama:** (Legal Term in India) A record of witness testimony, often used by police during investigations, detailing observations. In this context, the tool aids in creating a digital equivalent for online evidence.
- **PDF:** Portable Document Format.
- **OS:** Operating System.
- **Tkinter:** Python's standard GUI package.
- **PIL/Pillow:** Python Imaging Library (Fork).
- **ReportLab:** A Python library for creating PDFs.

2. System Analysis

2.1 Introduction Digital evidence from social media is increasingly crucial in investigations. However, capturing this volatile data accurately and defensibly presents challenges. Manual methods are inefficient and prone to errors or omissions. FeedLens is proposed as an automated solution to improve this process.

2.2 Existing System The current (manual) system typically involves:

- A. Investigator logging into the target social media account (often on a separate device or browser profile).
- B. Manually scrolling through timelines, messages, friend lists, etc.

- C. Using OS screenshot tools (like Snipping Tool, Print Screen) or browser extensions to capture relevant sections.
- D. Manually saving each screenshot with a potentially inconsistent naming convention.
- E. Manually copying/pasting relevant text or transcribing information.
- F. Potentially missing content due to inconsistent scrolling or oversight.
- G. Lack of automated timestamping directly on the image at the moment of capture.
- H. Lack of automated hashing for integrity verification.
- I. Manual compilation of captured data into a report or case file. *Limitations:* Highly time-consuming, high risk of human error (missing data, inconsistent capture), lack of automated metadata (timestamps, hashes), difficulty in documenting the *entire* visible state of a long page, potential inconsistencies between investigators.

2.3 Proposed System (FeedLens - based on provided code)

1. FeedLens automate several aspects of the existing system:
2. Provides a GUI to manage the capture process and settings.
3. Uses Selenium WebDriver to control a standard web browser (Chrome, Firefox, Edge).
4. Opens a browser instance and navigates to a specified URL (preset for common platforms or user-defined).
5. Allow the investigator to log in manually within the controlled browser.
6. Automates scrolling (Page Down/Page Up) at a configurable speed.
7. Takes full-viewport screenshots sequentially during scrolling or on demand.
8. Optionally overlays the current date/time onto each screenshot.
9. Saves screenshots in a structured manner within a session folder (named using Case Number + Timestamp) using configurable image formats (PNG/JPEG).
10. Calculates the SHA-256 hash of each saved screenshot file.
11. Optionally performs OCR (using Tesseract) on screenshots to extract text.
12. Maintains an activity log of significant actions performed by the tool.
13. Generates a detailed PDF report consolidating case information, settings, activity log, captured images, extracted text, and hashes.

2.4 Advantages of Proposed System

1. **Efficiency:** Drastically reduces the time required to capture extensive social media profiles or web pages.
2. **Consistency:** Ensures screenshots are taken systematically during scrolling.
3. **Reduced Error:** Minimizes human error associated with manual scrolling and saving.
4. **Automated Documentation:** Automatically generates timestamps (overlay), hashes, and organizes files.
5. **Verifiability:** File hashes help ensure the integrity of the captured screenshot files.
6. **Searchability:** OCR text allows captured content to be potentially searchable (within the report or if text is exported).
7. **Comprehensive Reporting:** Creates a single, consolidated PDF report suitable for case files.
8. **Session Management:** Organizes captures logically based on case and time.

2.5 Operating Environment:

1. **Operating System:** Desktop OS (Windows tested primarily due to Tesseract path check and style usage; macOS and Linux should work with appropriate WebDriver and Tesseract installations).
2. **Software:** Python 3.x environment, Selenium library, specific WebDriver executable (chromedriver, geckodriver, msedgedriver) matching the installed browser version, Tesseract-OCR engine (installed and path configured or in system PATH), Pillow library, ReportLab library.
3. **Hardware:** Standard desktop or laptop meeting the minimum requirements for running a modern web browser and Python scripts. Sufficient RAM (especially if OCR is used) and disk space are necessary.
4. **Network:** Active internet connection required to access target websites.
5. **Critical Gap:** The provided code *does not* include an Android version as requested in the problem statement. It is purely a desktop application relying on desktop browser automation.

2.6 Design & Implementation Constraints

1. **Web Page Structure Dependency:** Relies heavily on the target website's HTML structure and JavaScript behavior. Changes to website design (especially dynamic loading or scroll mechanisms) can break the automation.

2. **Login & CAPTCHA:** The tool does not automate logins or solve CAPTCHAs. The investigator must handle authentication manually within the controlled browser window. Complex login flows or security measures might hinder access.
3. **Dynamic Content Loading:** Infinite scroll or dynamically loaded content might require more sophisticated wait conditions than the current fixed `time.sleep()` to ensure content is fully rendered before capture.
4. **Platform Specifics:** The current implementation uses generic browser scrolling (Page Up/Down). It lacks deep, specific parsing logic for individual platforms (e.g., reliably finding the 'end' of a specific feed type, handling platform-specific pop-ups automatically, extracting structured data beyond OCR). Support for Telegram/WhatsApp web interfaces would require specific testing and potentially different interaction logic. Google Account parsing would be limited to what's shown on standard web interfaces (like Gmail, Drive) via the browser.
5. **OCR Accuracy:** OCR quality depends heavily on image resolution, layout complexity, language, and Tesseract's training data. Preprocessing helps but isn't foolproof.
6. **Resource Intensive:** Running a full browser instance, taking frequent screenshots, and performing OCR can be resource-intensive (CPU, RAM, Disk I/O).
7. **WebDriver Maintenance:** Requires keeping WebDriver executables updated to match browser updates.
8. **Error Handling:** While some error handling exists (WebDriver init, file saving, OCR), complex browser or network errors might require more robust handling.
9. **Single-Threaded GUI:** Tkinter runs in the main thread. Long operations (like report generation or potentially complex OCR) could freeze the GUI if not handled carefully (though scrolling uses a separate thread).
10. **Lack of Android Version:** This is a major constraint compared to the problem statement. Developing an Android version would require a completely different technology stack (e.g., using Android automation frameworks like UIAutomator or Appium) and approach.

3. Feasibility Study

3.1 Economic Feasibility

1. **Development Cost:** Primarily the time cost of developing and maintaining the Python script.
2. **Software Cost:** Low, as it relies on open-source components (Python, Tkinter, Selenium, Tesseract, ReportLab, Pillow).
3. **Operational Cost:** Minimal, mainly relating to investigator time for setup and operation.
4. **Benefit:** Significant potential time savings for investigators compared to manual methods, leading to indirect cost savings. Improved evidence quality can strengthen cases.
5. **Conclusion:** Economically feasible, offering good potential return on the investment of development time.

3.2 Technical Feasibility

1. **Core Technology:** The use of Selenium for browser automation, Pillow for image handling, Tesseract for OCR, and ReportLab for PDF generation are all well-established and technically proven technologies.
2. **Desktop Version:** The provided code demonstrates the technical feasibility of the core features on a desktop platform.
3. **Challenges:** Maintaining compatibility with evolving websites and browsers, handling anti-bot measures, and ensuring robust OCR are ongoing technical challenges but are manageable.
4. **Android Version:** Technically feasible using *different* tools and frameworks (e.g., Android Studio, UI Automator, Appium), but represents a separate, significant development effort *not* covered by the current code. The current code *cannot* be easily ported to Android.
5. **Conclusion:** The desktop version is technically feasible. The requested Android version requires a separate project/approach. The specific handling of all listed platforms (Telegram, WhatsApp, deep Google Account) needs further investigation and likely platform-specific code additions.

3.3 Operational Feasibility

1. **User Skill:** Requires investigators to have basic computer literacy, be comfortable installing software/drivers, and understand how to navigate the target websites.
2. **Workflow Integration:** Fits well into the investigative workflow where an examiner needs to document the state of a web account.
3. **Setup:** Requires initial setup of Python, libraries, WebDriver, and Tesseract, which might be a hurdle in restricted IT environments. Packaging the application (e.g., using PyInstaller) could simplify deployment.
4. **Training:** Minimal training should be required for the GUI, but users need to understand the tool's limitations (e.g., manual login, potential issues with dynamic content).
5. **Conclusion:** Operationally feasible for investigators with standard technical skills, provided the necessary prerequisites can be installed.

4. State of Art / Literature Review

4.1 Literature Review The field of digital forensics includes numerous tools and techniques for data acquisition.

1. **Commercial Forensic Suites:** Tools like Cellebrite UFED Cloud Analyzer, Magnet AXIOM Cloud, Oxygen Forensic Detective specialize in extracting data from cloud accounts, often using APIs, tokens, or specific forensic methods. These are typically comprehensive but expensive and may require specific expertise.
2. **Open-Source Intelligence (OSINT) Tools:** Various OSINT frameworks and scripts exist (e.g., Maltego, SpiderFoot, custom scrapers) that gather publicly available information or interact with social media APIs (where available and permitted). Their focus is often broader intelligence gathering rather than forensic documentation of a logged-in session.
3. **Web Scraping Libraries/Frameworks:** Python libraries like Scrapy and BeautifulSoup are powerful for extracting structured data from websites but usually require more custom coding for specific sites and don't inherently provide a GUI or the forensic documentation features (hashing, reporting) of FeedLens.

4. **Browser Automation:** Selenium (used here) and alternatives like Playwright or Puppeteer are standard for controlling browsers, enabling interaction with JavaScript-heavy sites, and taking screenshots.
5. **Screenshot Tools:** Numerous basic screenshot utilities exist, but lack the automation, scrolling, OCR, hashing, and reporting integration needed here.
6. **OCR Technology:** Tesseract is a widely used open-source OCR engine. Commercial OCR solutions also exist.

4.2 Research Gap While comprehensive forensic suites exist for cloud data extraction, and generic web scrapers/screenshot tools are available, FeedLens targets a specific niche:

1. An investigator-driven tool focused on *documenting the live view* of a web interface during examination, rather than solely relying on API extraction or logical file system analysis.
2. Combining automated *scrolling* capture (both up and down) with integrated timestamping, hashing, OCR, and PDF reporting within a relatively simple GUI package.
3. Providing a potentially lower-cost, more accessible alternative to full commercial suites for the specific task of documenting web interface content.
4. **Key Gap Addressed by Problem Statement (but not code):** The specific requirement for an *Android version* to handle scenarios where desktop logins fail highlights a significant gap not addressed by most standard desktop automation tools or the current FeedLens code. Similarly, dedicated handling routines for platforms like Telegram/WhatsApp (often app-focused) are less common in general web automation tools.

5. System Specifications

5.1 Hardware Requirements

1. **Processor:** Modern Multi-core CPU (e.g., Intel i3/i5/i7, AMD Ryzen 3/5/7 or equivalent).
2. **RAM:** Minimum 8 GB recommended (16 GB+ suggested for smoother operation, especially with OCR and many tabs).
3. **Storage:** Sufficient free disk space for:
 - Python installation (~100-200 MB).

- Installed libraries (~200-500 MB).
 - WebDriver executable (~10-100 MB per browser).
 - Tesseract-OCR installation (~50 MB - 1 GB+ depending on language packs).
 - Captured screenshots and reports (Variable, potentially GBs depending on usage). SSD recommended for faster file I/O.
4. **Display:** Standard monitor resolution.
 5. **Network:** Network Interface Card for internet access.

5.2 Software Requirements

1. **Operating System:** Windows 10/11 recommended. macOS or Linux distributions capable of running Python 3 and required drivers.
2. **Python:** Python 3.7 or later.
3. **Python Libraries:** selenium, pillow (PIL fork), reportlab, pytesseract, tkinter (usually included), queue, threading.
4. **Web Browser:** Recent version of Google Chrome, Mozilla Firefox, or Microsoft Edge.
5. **WebDriver:** Corresponding WebDriver executable (chromedriver, geckodriver, msedgedriver) matching the browser version, accessible in the system PATH or specified location.
6. **Tesseract-OCR:** Installed Tesseract engine (v4 or v5 recommended). Path to tesseract.exe configured in the script or added to system PATH. Required language data packs installed.

6. Specific & Functional Interfaces

6.1 User Interfaces

1. Main application window (Tkinter based).
2. Menu bar (File, Help).
3. Tabbed interface (ttk.Notebook):
 - **Capture & Report Tab:** Case Information inputs (Case Number, Investigator, Notes), Action buttons (Open/Close Browser, Single Shot, Start Scroll Down/Up, Stop), Reporting buttons (Generate PDF, Open Folder).

- **Settings Tab:** Configuration options for Browser, Platform Preset, Base Output Directory, Scroll Delay, Stagnation Threshold, Image Format, Timestamp Overlay, OCR Enable/Language. Browse button for directory selection.
 - **Activity Log Tab:** Scrolled text area displaying timestamped logs of tool operations. Save Log button.
4. Status bar: Displays current status messages, screenshot count, progress bar.
 5. Dialog boxes (messagebox, filedialog, simpdialog, Toplevel): For errors, warnings, confirmations, file selection, URL input, report options.

6.2 Hardware Interfaces

Interacts with standard PC input/output hardware: Keyboard, Mouse, Display, Storage Drives (HDD/SSD), Network Adapter. No specialized hardware required.

6.3 Software Interfaces

1. **Operating System:** Uses os and sys modules for path manipulation, file system operations (creating directories, saving files, checking existence), platform detection, and subprocess / os.startfile to open file explorer.
2. **Web Browser:** Interacts via the Selenium WebDriver API to control browser actions (launch, navigate, execute JavaScript, capture screenshots, send keys like PageUp/PageDown).
3. **Tesseract-OCR:** Executes the Tesseract command-line tool via pytesseract wrapper to perform OCR on image files.
4. **Python Standard Library:** Uses datetime, time, hashlib, re, queue, threading, pathlib.
5. **Third-Party Libraries:** Uses tkinter/ttk (GUI), selenium (Browser Automation), PIL/Pillow (Image Processing for OCR), reportlab (PDF Generation).

6.4 Communication Interfaces

Uses standard web protocols (HTTP/HTTPS) implicitly through the controlled web browser to communicate with target websites. Does not implement direct network socket communication itself.

7. Project Description

7.1 Problem Definition The core problem is the inefficiency, potential for error, and lack of standardization in manually capturing digital evidence from social media and web interfaces during investigations. Investigators need a reliable way to document the state of accounts (timelines, messages, follower lists, etc.) as observed, often requiring extensive scrolling and numerous screenshots. Key requirements include automated capture, timestamping, hashing for integrity, text extraction (OCR), consolidated reporting, support for major platforms (Facebook, X, Instagram, potentially others like Telegram, WhatsApp, Google Accounts), and the ability to function both on desktops and potentially Android devices (due to login issues sometimes encountered on desktops).

7.2 Project System Overview FeedLens is a Python-based desktop application designed to automate the capture of web content for investigative documentation. It presents a GUI allowing users to input case details, configure settings (browser, output location, capture parameters), and control the capture process. Using Selenium WebDriver, it launches and controls a browser instance. After the user manually logs into the target site, FeedLens can take single screenshots or automatically scroll up/down the page, capturing sequential screenshots at defined intervals. It saves these images (optionally with timestamp overlays) to a session-specific folder, calculates SHA-256 hashes, and can perform OCR to extract text. Finally, it compiles selected information (case details, settings, logs, images, text, hashes) into a landscape-oriented PDF report.

7.3 Modules Overview & UML Analysis Models

Modules Overview (Logical Components based on Code):

1. **GUI Module (FeedLensApp, Tkinter Widgets):** Handles all user interaction, displays status, manages input fields, buttons, tabs, and dialogs. Orchestrates calls to other modules.
2. **Session Management Module:** Handles creation and tracking of the active output directory based on case number and timestamp. Manages session-specific data like screenshot lists and hashes.
3. **Browser Automation Module (init_driver, Selenium Integration):** Manages WebDriver instances (creation, closing), browser navigation, JavaScript execution (for timestamps), and sending scroll commands.

4. **Screenshot Capture Module (capture_screenshot, _auto_scroll logic):** Takes screenshots using WebDriver, saves them to the session folder, handles timestamp overlays, triggers hashing. Manages the automated scrolling loop and stagnation detection in a separate thread.
5. **Hashing Module (get_sha256_hash, hashlib):** Calculates SHA-256 hashes for generated files.
6. **OCR Module (extract_text, preprocess_image, Pytesseract Integration):** Preprocesses images (grayscale, contrast, etc.) and invokes Tesseract OCR engine to extract text.
7. **Reporting Module (generate_report, ReportLab Integration):** Gathers data based on user selection (case info, settings, logs, images, text, hashes) and builds the final PDF document using ReportLab library elements (Paragraphs, Tables, Images, Spacers).
8. **Logging Module (log_activity):** Records timestamped events and messages to an internal list and updates the GUI log view. Supports saving the log to a file.
9. **Configuration Module (Tkinter Variables):** Holds and manages application settings via Tkinter variables (StringVar, IntVar, etc.).
10. **Utility Module (sanitize_filename, open_file_explorer):** Provides helper functions for filename cleaning and opening folders in the OS file explorer.

UML Analysis Models (Conceptual):

1 Use Case Diagram:

- a. **Actor:** Investigator
- b. **Use Cases:**
 - i. Configure Settings (Browser, Output Dir, Scroll, OCR, etc.)
 - ii. Enter Case Information
 - iii. Start New Browser Session (Includes folder creation)
 - iv. Manually Authenticate (External to direct tool control)
 - v. Capture Single Screenshot
 - vi. Start Automated Scroll Capture (Down)
 - vii. Start Automated Scroll Capture (Up)
 - viii. Stop Scroll Capture
 - ix. Generate PDF Report (with options)
 - x. View Activity Log

- xi. Save Activity Log to File
- xii. Open Session Output Folder
- xiii. Close Browser Session
- xiv. Exit Application

Class Diagram (Simplified):

1. FeedLensApp (Main Application Class)

- a. *Attributes:* root (Tkinter), driver (WebDriver), screenshot_paths (List), extracted_texts (List), image_hashes (Dict), case_number_var (StringVar), settings_vars..., active_output_dir (Path), activity_log (List), etc.
- b. *Methods:* __init__, create_widgets..., open_browser, close_browser, capture_screenshot, start_screenshot, stop_screenshot, _auto_scroll, extract_text, generate_report, log_activity, update_status, on_exit, etc.

(External Libraries/Classes Used):

- 1. selenium.webdriver (Chrome, Firefox, Edge)
- 2. tkinter (Root, Frame, Button, Entry, etc.)
- 3. reportlab.platypus (SimpleDocTemplate, Paragraph, Image, Table, etc.)
- 4. PIL.Image
- 5. pytesseract

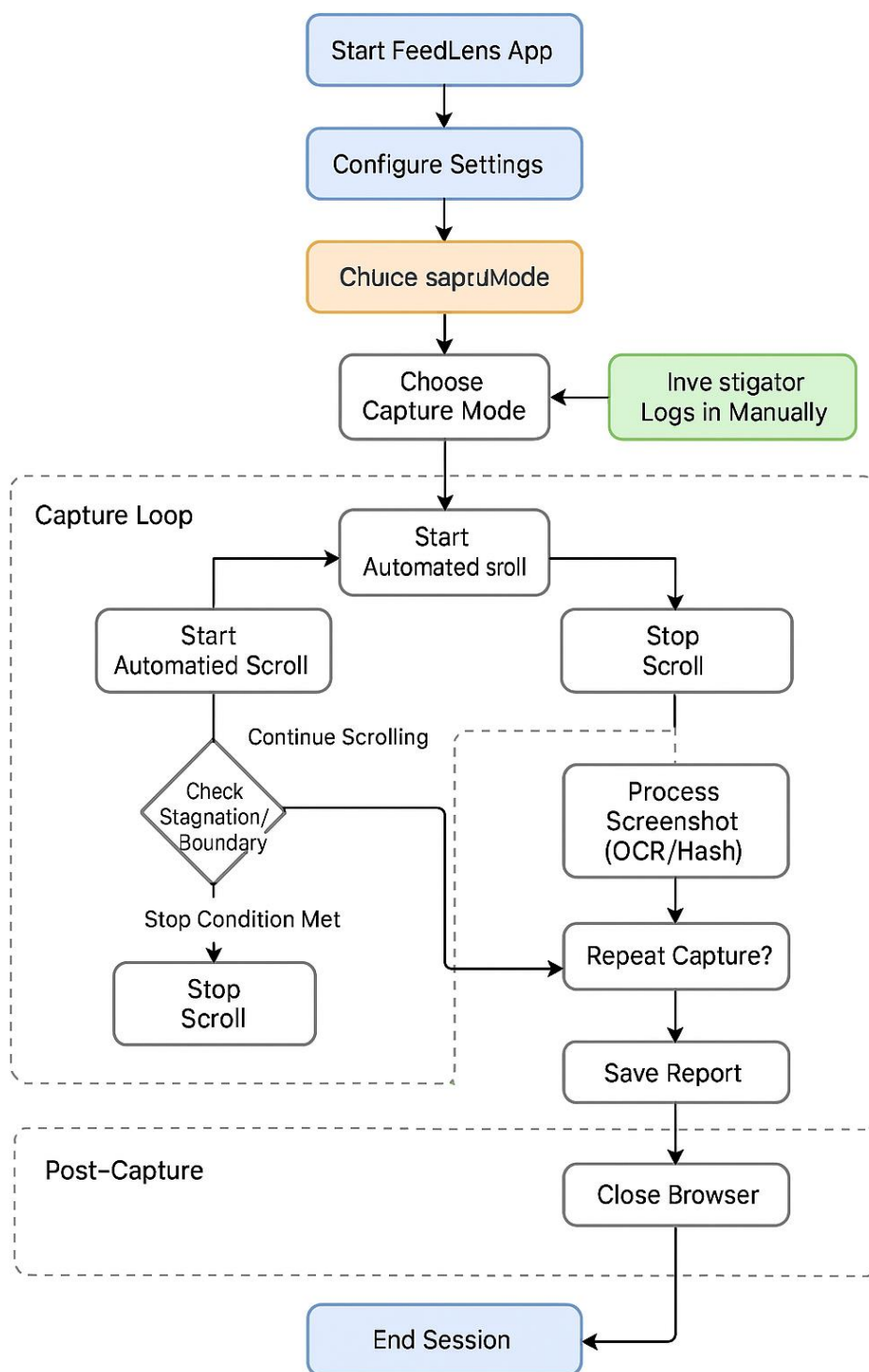
Activity Diagram (Simplified: Automated Scroll Capture):

- 1. [Start] -> Investigator Clicks "Start Scroll Down/Up"
- 2. FeedLensApp: Update Button States (Disable irrelevant, Enable Stop)
- 3. FeedLensApp: Update Status Label
- 4. FeedLensApp: Start Background Thread (_auto_scroll)
- 5. **Background Thread:**
 - a. Get Initial Scroll Position
- 6. **Loop (While running_down/running_up is True and no errors):**
 - a. capture_screenshot -> (Save PNG/JPEG, Inject/Remove Timestamp, Calculate Hash)
 - b. If Screenshot OK: * Append path to screenshot_paths * extract_text (if OCR enabled) -> Append text to extracted_texts * Put updated count in count_queue

- c. Else (Screenshot Failed): Log Error, Break Loop
 - d. Send Scroll Key (Page Down/Up)
 - e. Wait (scroll_delay)
 - f. Get Current Scroll Position
 - g. Check for Stagnation / Page Boundary
 - h. If Stagnant/Boundary Threshold reached: Log Reason, Break Loop ix.
Else: Reset Stagnation Count
- 7. End Loop**
- 8. Background Thread:**
- a. Schedule stop_screenshot call on Main Thread
- 9. Main Thread (_poll_queue running periodically):**
- a. Get count from count_queue -> update_status (GUI count, progress bar)
- 10. Main Thread (stop_screenshot called):**
- a. Set running_down/running_up to False
 - b. Log Stop
 - c. Update Status Label
 - d. Update Button States (Enable relevant actions)
 - e. Clear count_queue

8. System Design Diagrams

8.1 System Flow Diagrams



The diagram illustrates the FeedLens System Processes, showing the interaction between External Entities, the FeedLens System Processes, and Data Stores.

External Entities:

- Investigator
- Target Website
- Tesseract OCR Engine

FeedLens System Processes:

- 1.0 Manage Session & Config:** Receives Case Info, Config Settings and Control Commands from the Investigator. It outputs Validated Settings to the Settings data store.
- 2.0 Control Browser:** Receives Navigation/Scroll Cmd from the Target Website and Raw Screenshot from the 4.0 Generate Report process. It outputs Raw Screenshot and Raw OCR to the 4.0 Generate Report process.
- 4.0 Generate Report:** Receives Raw Screenshot and Raw OCR from the 2.0 Control Browser. It outputs Capture Status to the 5.0 Manage Activity Log process and PDF File to the 5.0 Manage Activity Log process.
- 5.0 Manage Activity Log:** Receives Capture Status and PDF File from the 4.0 Generate Report process. It outputs Report Status to the 4.0 Generate Report process and Log File to the Activity Log data store.
- 4.0 Generate Report (bottom):** Receives Case Info, Notes from the Investigator and Display Log from the Target Website. It outputs Save Log Request to the 5.0 Manage Activity Log process and PDF Report to the Settings data store.

Data Stores:

- Settings:** Receives Validated Settings from the 1.0 Manage Session & Config process and PDF Report from the 4.0 Generate Report (bottom) process.
- Session Data (Screenshots, hah) Text):** Receives Log Entry from the Settings data store.
- Activity Log:** Receives Log File from the 5.0 Manage Activity Log process.

```

    usecaseDiagram
        participant ExternalEntities as External Entities
        participant FeedLensSystem as FeedLens System Processes
        participant DataStores as Data Stores

        ExternalEntities --> 1.0 : Case Info, Config Settings
        ExternalEntities --> 2.0 : Control Commands
        ExternalEntities --> 3.0 : Save Log Request

        1.0 --> 2.0
        2.0 --> 3.0
        3.0 --> 4.0
        4.0 --> 5.0
        5.0 --> 5.0

        1.0 --> SDH : Raw Screenshot
        3.0 --> SDH : Screenshot
        4.0 --> SDH : Report Status

        SDH --> Settings
        Settings --> SDH : Screenshot
        SDH --> AL : PDF Report
        AL --> SDH : Activity Log

        4.0 --> Tesseract1 : Screenshot Count
        Tesseract1 --> 4.0 : Count
        4.0 --> Tesseract2 : Screenshot Count
        Tesseract2 --> 4.0 : Count
        4.0 --> 5.0 : Screenshot Count
        5.0 --> 5.0 : Screenshot Count

        5.0 --> DisplayLog : Display Log
        DisplayLog --> 5.0
        DisplayLog --> 5.0
    
```

The diagram illustrates the FeedLens System Processes, categorized into External Entities, FeedLens System Processes, and Data Stores.

External Entities:

- Investigator
- Target Website
- File System
- Tesseract OCR Engine (two instances)

FeedLens System Processes:

- 1.0 Manage Session & Config
- 2.0 Control Browser
- 3.0 Capture & Process Data
- 4.0 Generate Report
- 5.0 Manage Activity Log
- 5.0 Report Status

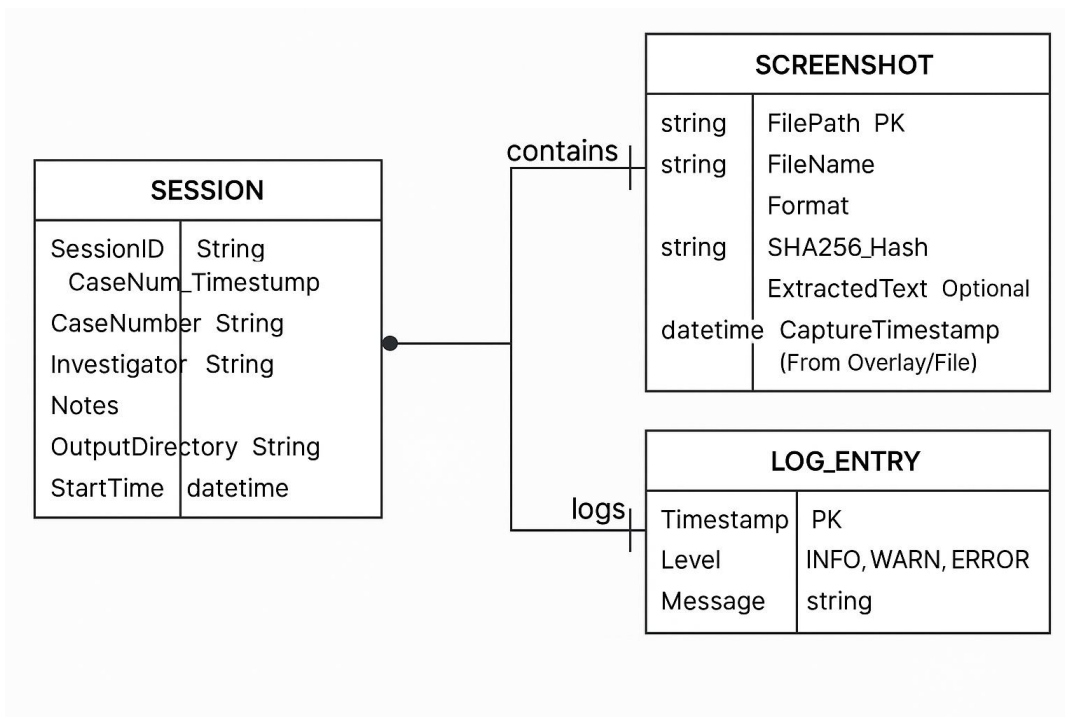
Data Stores:

- Settings
- Session Data Hashes, Text
- Activity Log

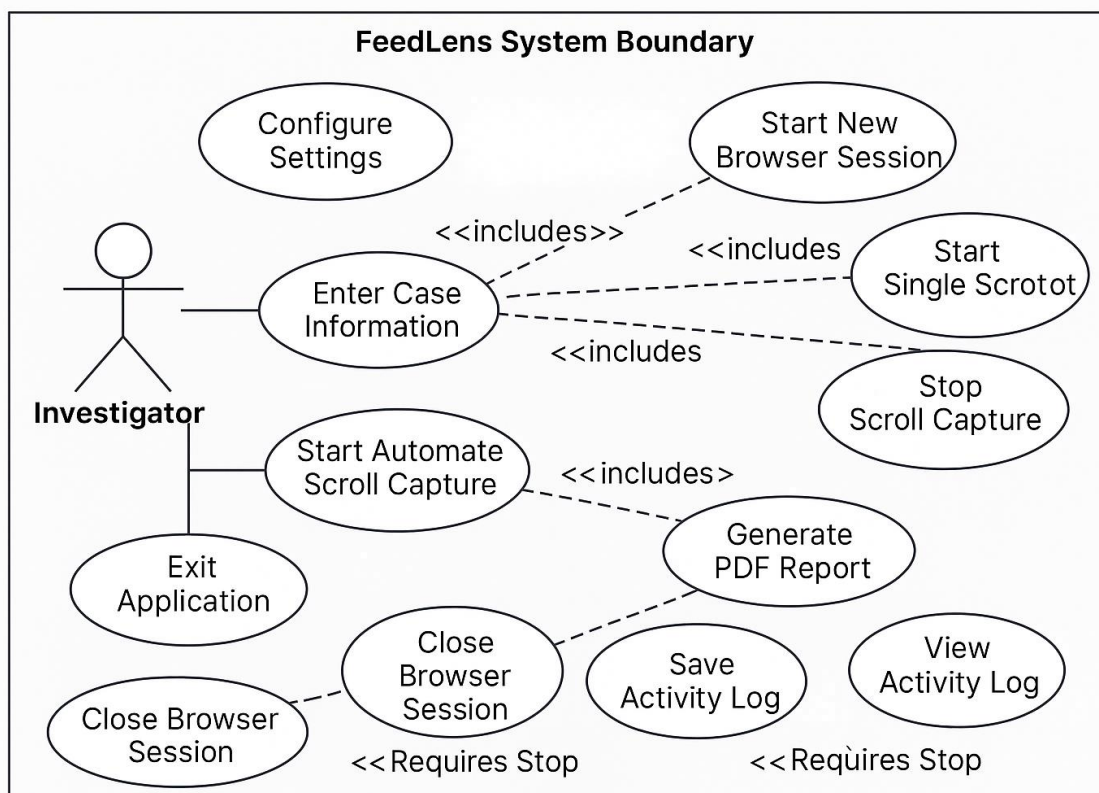
Interactions:

- Investigator** sends **Case Info, Config Settings** to **1.0 Manage Session & Config**.
- Target Website** sends **Control Commands** to **2.0 Control Browser**.
- File System** sends **Save Log Request** to **3.0 Capture & Process Data**.
- 1.0 Manage Session & Config** sends **Raw Screenshot** to **Session Data Hashes, Text**.
- 2.0 Control Browser** sends **Screenshot** to **Session Data Hashes, Text**.
- 3.0 Capture & Process Data** sends **Report Status** to **Session Data Hashes, Text**.
- Session Data Hashes, Text** sends **Screenshot** to **Settings**.
- Settings** sends **Screenshot** to **Session Data Hashes, Text**.
- Session Data Hashes, Text** sends **PDF Report** to **Activity Log**.
- Activity Log** sends **Activity Log** to **Session Data Hashes, Text**.
- 4.0 Generate Report** sends **Screenshot Count** to **Tesseract OCR Engine** (left).
- Tesseract OCR Engine** (left) sends **Count** to **4.0 Generate Report**.
- 4.0 Generate Report** sends **Screenshot Count** to **Tesseract OCR Engine** (right).
- Tesseract OCR Engine** (right) sends **Count** to **4.0 Generate Report**.
- 4.0 Generate Report** sends **Screenshot Count** to **5.0 Manage Activity Log**.
- 5.0 Manage Activity Log** sends **Screenshot Count** to **5.0 Report Status**.
- 5.0 Report Status** sends **Display Log** to **5.0 Manage Activity Log**.
- 5.0 Manage Activity Log** sends **Display Log** to **5.0 Report Status**.

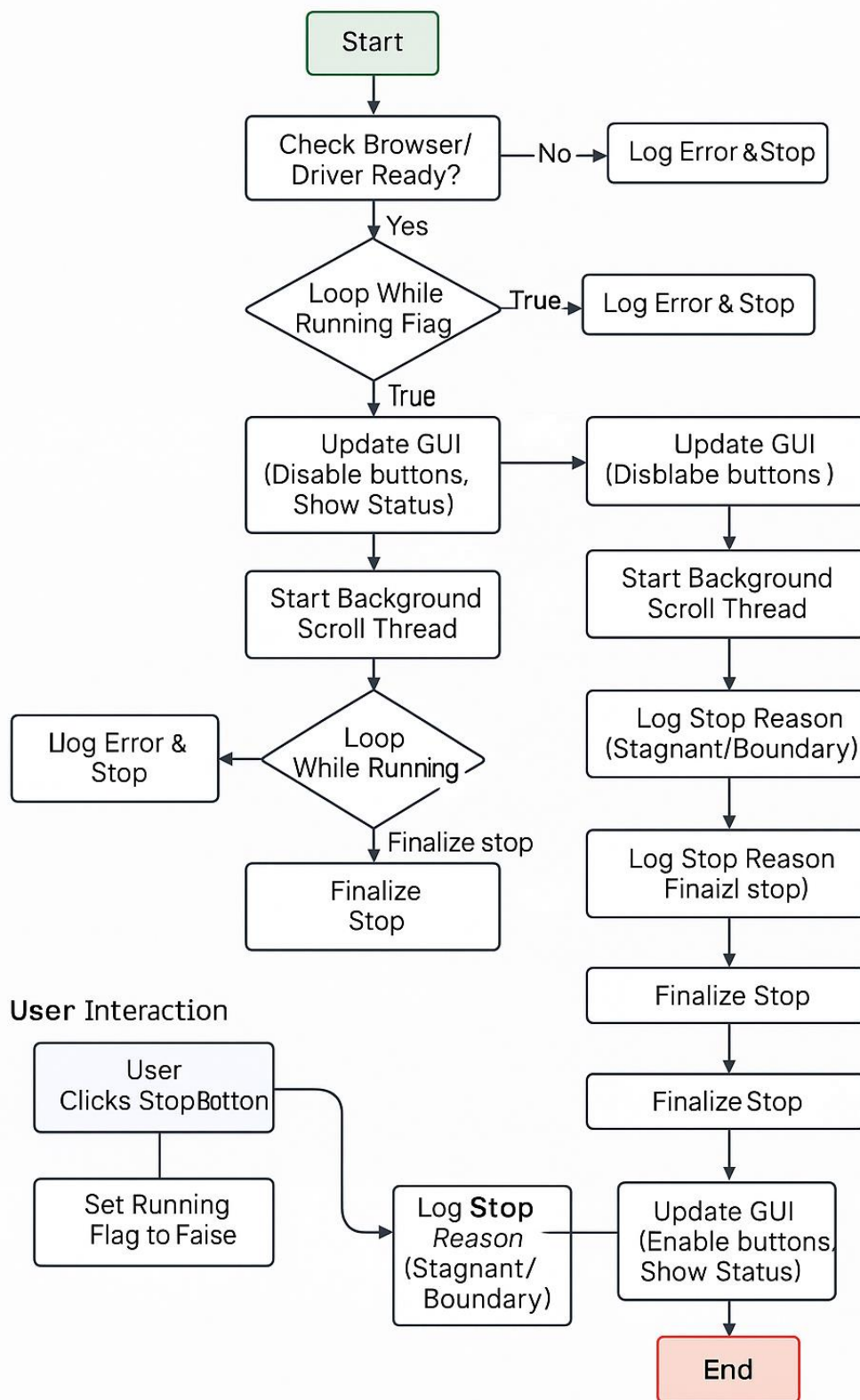
8.4 Entity Relationship Diagrams (ERDs)



8.5 Use Case Diagram



8.6 Activity Diagrams



9. FeedLens – Potential Future Enhancements

The potential future enhancements for the FeedLens tool, aiming to improve its capabilities, address current limitations, and align more closely with the original problem statement and evolving digital investigation needs.

9.1 Android Version Development

Requirement:

Address the critical need for an Android version, especially for scenarios where desktop access fails or mobile app interfaces are the primary investigative targets.

Implementation Approach:

- Develop a dedicated Android application using Android-native frameworks (e.g., **UIAutomator**, **Espresso**) or cross-platform tools like **Appium**.
- Key components:
 - Build an Android app interface to initiate and manage capture tasks.
 - Use accessibility services or UI automation to interact with third-party apps (Facebook, Instagram, WhatsApp, Telegram).
 - Implement mechanisms for:
 - Automated screenshot capture.
 - Scroll simulation to capture dynamic content.
 - Data extraction (text/images).
 - Enable secure data transfer (images, logs, extracted text) to a centralized repository or support report generation directly on the device.
 - Navigate Android security policies (e.g., permissions management, accessibility compliance).

9.2 Enhanced Platform-Specific Handling

Requirement:

Move beyond basic web scrolling to more intelligent, context-aware interactions tailored to specific platforms.

Implementation Approach:

- **Element-Specific Targeting:** Utilize robust selectors (CSS/XPath) to accurately identify and interact with key sections like:
 - Facebook posts and friend lists.
 - Instagram grids and stories.
 - Twitter/X timelines.
 - Messaging panels on WhatsApp Web and Telegram Web.
- **Smart Scrolling & Pagination:** Detect and interact with "Load More" buttons, infinite scroll endpoints, and dynamic content loads instead of relying solely on fixed scrolling delays.
- **Targeted Capture Options:** Allow users to specify capture goals, such as:
 - Full friend lists.
 - Timeline posts between specific dates.
 - Direct messages from/to selected users.
- **Metadata Extraction:** Parse structured HTML data (timestamps, author names, URLs) to supplement OCR outputs.
- **Dedicated Platform Modules:** Develop modular plugins for each major platform to encapsulate and maintain platform-specific logic separately.

9.3 Advanced OCR & Text Processing

Requirement:

Increase the accuracy, flexibility, and depth of extracted text data.

Implementation Approach:

- **Preprocessing Enhancements:** Apply techniques like adaptive thresholding, de-skewing, and noise reduction for cleaner OCR inputs.

- **Dynamic Layout Handling:** Experiment with Tesseract's Page Segmentation Modes (PSM) to optimize for varying content layouts.
- **Multi-Language OCR Support:** Enable automatic detection or manual selection of multiple OCR languages; simplify management of Tesseract language packs.
- **Post-OCR Corrections:** Integrate spell-checkers and context-aware correction systems to improve text quality.
- **Named Entity Recognition (NER):** Apply NLP techniques to tag entities like names, locations, dates, and organizations, enriching extracted data for investigations.

9.4 AI-Powered Data Summarization & Analysis

Requirement:

Help investigators quickly understand large volumes of captured text by providing intelligent summaries and insights.

Implementation Approach:

- **LLM Integration:** Connect to cloud-based APIs (e.g., OpenAI, Google Gemini) or support local deployment of LLMs for enhanced data privacy.
- **Summarization Features:**
 - Allow investigators to send full sessions or selected text segments for AI-driven summarization.
 - Enable adjustable summary focus (e.g., by topics, conversations, keywords).
- **Sentiment Analysis:** Provide optional sentiment evaluation for posts, messages, or interactions.
- **Report Integration:** Embed AI-generated summaries or analysis sections into the final PDF or HTML reports, clearly separated and labeled.
- **Privacy Controls:** Offer clear warnings and options regarding cloud services usage, prioritizing local processing where possible.

9.5 Reporting & Export Enhancements

Requirement:

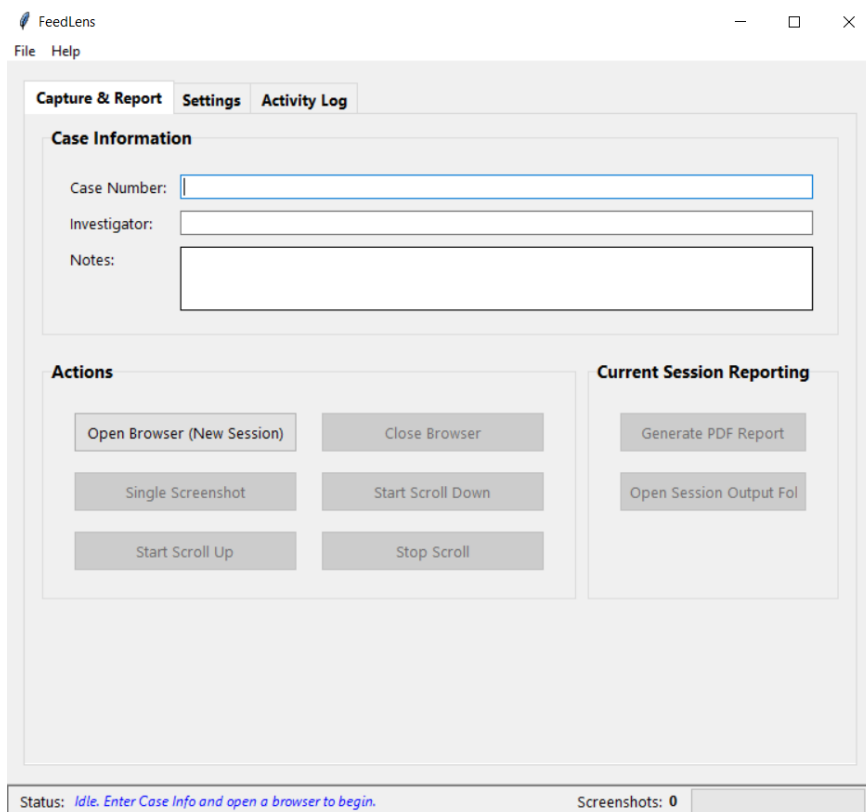
Deliver more flexible, customizable, and investigator-friendly reporting outputs.

Implementation Approach:

- **Customizable Templates:** Allow users to design or select report templates based on case type or personal preference.
- **Multiple Export Formats:**
Support export to HTML, CSV (for structured/tabular data like friend lists), JSON, XML, and traditional PDFs.
- **Interactive Reports:**
Generate HTML reports featuring searchable text, embedded screenshots, and interactive linking.
- **Report Annotations:**
Provide tools for investigators to add annotations or notes directly linked to specific screenshots, text blocks, or metadata within reports.

10. System Design Screen & Outputs

10.1 System UI



FeedLens

File Help

Capture & Report Settings Activity Log

Case Information

Case Number:

Investigator:

Notes:

Actions

Open Browser (New Session) Close Browser

Single Screenshot Start Scroll Down

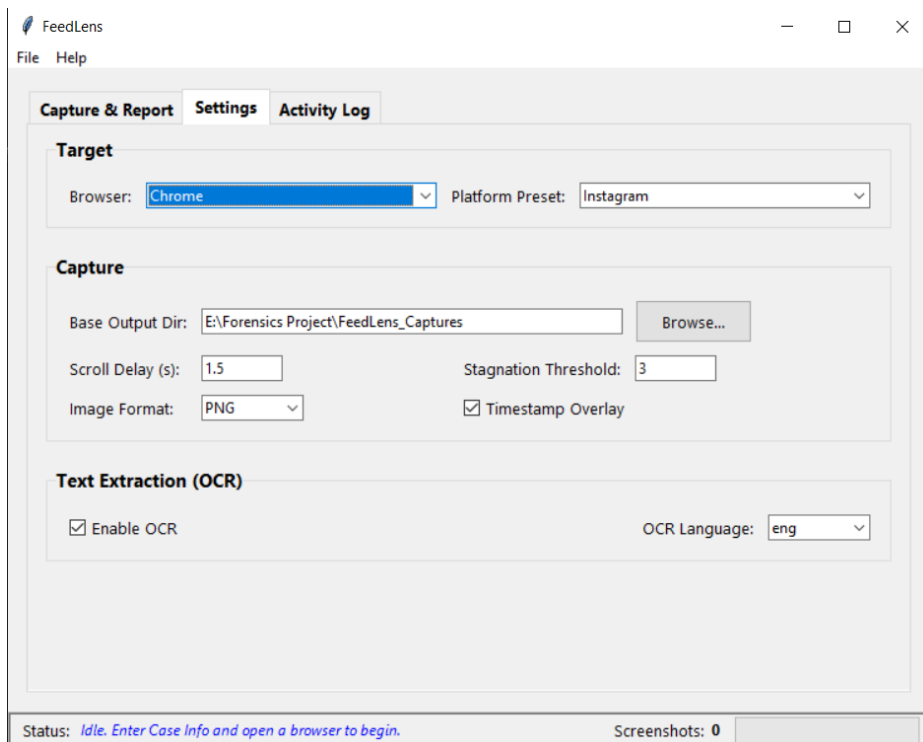
Start Scroll Up Stop Scroll

Current Session Reporting

Generate PDF Report

Open Session Output Fol

Status: *Idle. Enter Case Info and open a browser to begin.* Screenshots: 0



FeedLens

File Help

Capture & Report Settings Activity Log

Target

Browser: Chrome Platform Preset: Instagram

Capture

Base Output Dir: E:\Forensics Project\FeedLens_Captures Browse...

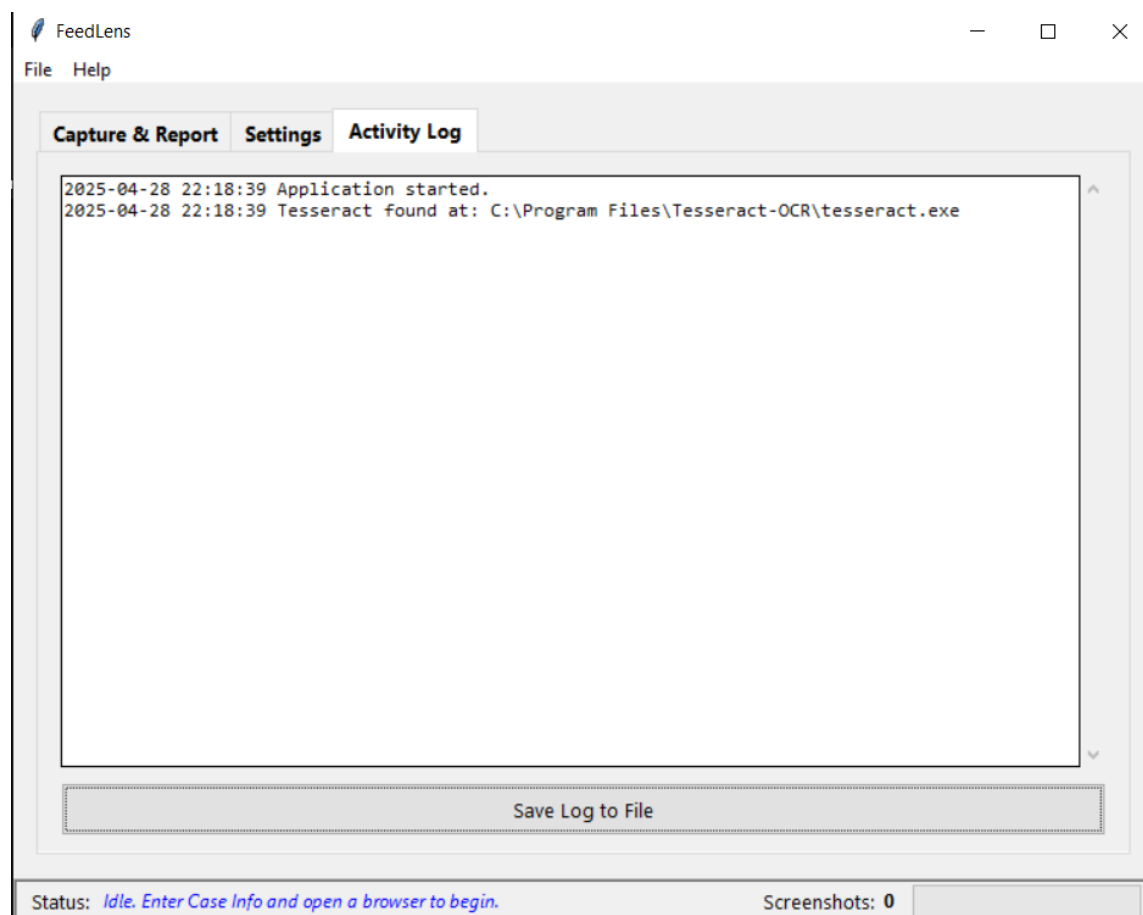
Scroll Delay (s): 1.5 Stagnation Threshold: 3

Image Format: PNG ☒ Timestamp Overlay

Text Extraction (OCR)

☒ Enable OCR OCR Language: eng

Status: *Idle. Enter Case Info and open a browser to begin.* Screenshots: 0



10.2 Outputs

Captured Screenshots & Data

Screenshot 1: single_X_20250425_224229.png

Image:	The screenshot shows a social media interface with a dark theme. On the left is a navigation menu with options: Home, Explore, Notifications, Messages, Grok, Communities, Premium, Verified Orgs, Profile, and More. The main feed area displays a large advertisement for "BC.GAME" with the text "FREE USDT, CRAZY BONUSES - \$686,982" and a "PLAY NOW" button. To the right of the feed are sections for "Trending" topics (Nuclear, Mohanlal, AK 47, Team Gujarat) and "Who to follow" (Microsoft Power, Sachin Tendulkar, Microsoft Outlook). At the bottom of the feed, a post by "Samrat Mandal" is partially visible.
SHA-256:	3746aee6b76c6041ec92cade799d63514f89db06228e5cdc2605cb1fea914289

Enhancing Social Media Evidence Collection and Reporting – FeedLens

Extracted Text:	<pre> a 2025-04-25 22:42:28 xX For you Following Q Search fA Home Bb BC.GAME @ @bcgame Ado Politics - Trending wee vcaauu Sign Up & Get UP TO\$20,000 in Casino or Sports! Click to Win the fortune Nuclear Q_ Explore bit ly/4jRcSAK 81.9K posts land Notifications Entertainment - Trending _ J Mohanlal (3 BC.GAME cre s) Messages FREEUSDY,CRAZY =," - Grok ' a AK47 BONUSES - \$686,982 ae oo wae Li Co Communities - Trending in India ws 4 pom a\ Team Gujarat X. Premium a y a. c ~ Show more Ad ~ age o~ > oe J Y Verified Orgs S LN i * . iif, Who to follow a Profile AL " Microsoft Power... @ Hil r Fotiow) " Foll @ More "ia r By @MsPowerBI crow : xY Sachin Tendulkar @ t Fotow Post PLAY NOW Re 78) Sachinter Follow Sign Up & Get UP TO\$20,000 \$1 Sigr @Outlook _ j Samrat Mandal wee @ 622 t1 920 @ 8k i 30M As S 7 @Samrat_SM9gs3 A Recently pinned Show more -- </pre>
------------------------	---

Activity Log

```

2025-04-25 22:35:29 [INFO] Application started.
2025-04-25 22:35:29 [INFO] Tesseract found at: C:\Program Files\Tesseract-OCR\tesseract.exe
2025-04-25 22:38:18 [INFO] Created new session output directory: E:\Forensics Project\FeedLens_Captures\NoCase_20250425_223818
2025-04-25 22:38:18 [INFO] --- New Session Started ---
2025-04-25 22:38:18 [INFO] Active output directory set to: E:\Forensics Project\FeedLens_Captures\NoCase_20250425_223818
2025-04-25 22:38:18 [INFO] Initializing Chrome WebDriver...
2025-04-25 22:38:19 [INFO] Chrome WebDriver initialized successfully.
2025-04-25 22:38:19 [INFO] Navigating to: https://www.instagram.com
2025-04-25 22:38:25 [INFO] Successfully navigated to https://www.instagram.com
2025-04-25 22:40:51 [INFO] Closing browser...
2025-04-25 22:40:53 [INFO] Browser closed successfully.
2025-04-25 22:41:04 [INFO] Created new session output directory: E:\Forensics Project\FeedLens_Captures\7_20250425_224104
2025-04-25 22:41:04 [INFO] --- New Session Started ---
2025-04-25 22:41:04 [INFO] Active output directory set to: E:\Forensics Project\FeedLens_Captures\7_20250425_224104
2025-04-25 22:41:04 [INFO] Initializing Chrome WebDriver...
2025-04-25 22:41:05 [INFO] Chrome WebDriver initialized successfully.
2025-04-25 22:41:05 [INFO] Navigating to: https://www.instagram.com
2025-04-25 22:41:10 [INFO] Successfully navigated to https://www.instagram.com
2025-04-25 22:41:11 [INFO] Closing browser...
2025-04-25 22:41:13 [INFO] Browser closed successfully.
2025-04-25 22:41:42 [INFO] Created new session output directory: E:\Forensics Project\FeedLens_Captures\7_20250425_224142
2025-04-25 22:41:42 [INFO] --- New Session Started ---
2025-04-25 22:41:42 [INFO] Active output directory set to: E:\Forensics Project\FeedLens_Captures\7_20250425_224142
2025-04-25 22:41:42 [INFO] Initializing Chrome WebDriver...
2025-04-25 22:41:44 [INFO] Chrome WebDriver initialized successfully.
2025-04-25 22:41:44 [INFO] Navigating to: https://twitter.com
2025-04-25 22:41:54 [INFO] Successfully navigated to https://twitter.com
2025-04-25 22:42:30 [INFO] Capturing screenshot: single_X_20250425_224229.png
2025-04-25 22:42:30 [INFO] Screenshot saved: E:\Forensics Project\FeedLens_Captures\7_20250425_224142\single_X_20250425_224229.png
2025-04-25 22:42:30 [INFO] SHA-256 Hash: 3746aee6b76c6041ec92cade799d63514f89db06228e5cdc2605cb1fea914289
2025-04-25 22:42:30 [INFO] Preprocessing image for OCR: single_X_20250425_224229.png
2025-04-25 22:42:34 [INFO] Performing OCR (lang=eng, config='--oem 3 --psm 6')...
2025-04-25 22:42:35 [INFO] OCR completed in 0.67s. Text length: 852 chars.
2025-04-25 22:42:51 [INFO] Starting PDF report generation...
2025-04-25 22:42:53 [INFO] Adding Case Info to report.
2025-04-25 22:42:53 [INFO] Adding screenshot details to report.
2025-04-25 22:42:53 [INFO] Adding Activity Log to report.

```

11. Conclusion

The FeedLens project addresses a critical challenge in modern digital investigations: the efficient, accurate, and forensically sound documentation of information from social media platforms and other web interfaces. Traditional manual methods—capturing screenshots, transcribing text, and compiling reports—are time-consuming, error-prone, and often lack standardization and verifiable integrity.

The development of the FeedLens desktop application, leveraging Python with Tkinter and integrated OCR technologies, provides a strong and valuable foundation for addressing these challenges. It offers investigators a streamlined approach to capturing and organizing digital evidence with improved accuracy and efficiency.

Looking ahead, future enhancements such as the development of an Android counterpart, the implementation of platform-specific parsing logic, the advancement of OCR capabilities, and the integration of AI-driven data summarization hold immense potential. These improvements could elevate FeedLens from a useful utility into a comprehensive, indispensable tool for digital investigators — significantly enhancing the collection, analysis, and reporting of web-based evidence while maintaining a rigorous focus on data integrity and forensic soundness.

With continued development, FeedLens has the opportunity to become a cornerstone solution for modern digital evidence gathering and analysis